Test and validation plan (template)

Replace everything in angle brackets. If your client has its own template, use that and borrow only the framework-specific wording, such as how evidence is produced.

Field	Value
System under test	[name and version]
Document ID	[ID from the client's QMS]
Version and date	[1.0, YYYY-MM-DD]
Author	[name, role]
Reviewer / approver	[name, role]

1. Purpose and scope

<What is being validated, and why. What is in scope (functions, interfaces, environments) and what is explicitly out of scope.>

2. System description

<Short description of the system, its users and roles, interfaces, and the data it handles. Mention whether it processes personal, financial or patient data.>

3. Requirements and risk

The requirements are listed in `requirements.json` with an ID, a title and a risk level. Each requirement is linked to the tests that verify it, and the traceability matrix shows the result.

Risk	Meaning	Minimum testing expected
High	[failure affects safety, money, compliance or data integrity]	[for example UI and API, negative cases]
Medium	[failure affects a main function but has a workaround]	[for example API plus one UI path]
Low	[cosmetic or convenience]	[for example smoke only]

[State how the risk level of each requirement was decided, and who agreed it.]

4. Test strategy

- **Approach:** automated end-to-end tests with Playwright and TypeScript, at the UI and API level.
- **Suites:** `@smoke` for critical paths, `@regression` for the wider suite, plus accessibility, visual, mobile and database checks where they apply.
- **Verification beyond the screen:** results are confirmed through the API and, where needed, the database (read-only), so a test does not pass on what the page says alone.
- **Negative and boundary cases:** [list the ones that matter for this system].
- **Known defects:** a defect in the system under test is recorded as a `@known-issue` test and handled as a deviation (section 9).
- **Out of scope for automation:** [for example visual judgement, exploratory testing, performance].

5. Environments and tooling

- **Test environment:** [URL, version, how it is built, who controls it].
- **Environment qualification:** the run manifest records the commit, tool versions, operating system, lock-file hash and application image digests. Compare them with the approved baseline before relying on a run.
- **Tools:** Playwright [version], Node.js [version], [others]. Fitness for purpose: <short justification in proportion to risk, for example open-source tool, widely used, outputs verified against the system>.
- **Where tests run:** [GitHub-hosted runner / self-hosted runner / client network].

6. Test data

<Synthetic or masked data only. Describe how it is created (builders, API) and removed (self-cleaning fixtures), and confirm that no real personal, financial or patient data is used.>

7. Roles

Role	Name	Responsibility
Test automation lead	<>	Writes and maintains tests, executes and reports
Reviewer	<>	Reviews test changes and results
QA approver	<>	Approves the plan and the summary report
System owner	<>	Confirms requirements and accepts residual risk

8. Acceptance criteria

- Every high-risk requirement is verified, and no requirement is failing.
- No requirement has the verdict "Not tested", unless it is listed here with a reason: <>.
- Every deviation is recorded, assessed and approved (section 9).
- The evidence pack is complete and its checksums verify.

9. Deviations and defects

- A failed test is investigated and recorded before the run is accepted. The cause is classified as a defect in the system, a defect in the test, or an environment problem.
- A defect in the system that is accepted for now is written as a `@known-issue` test and listed in the summary report with its risk assessment and a plan.
- A flaky test is not accepted as a pass. It is fixed or removed under change control.

10. Change control

Changes to tests, the requirement list or the pipeline go through a pull request with a reviewer. Branch protection and code owners are set as described in `repository-controls.md`. The commit that was validated is the one named in the evidence manifest.

11. Records and retention

The evidence pack of the approved run is stored in [system / location] for [period]. It contains the traceability matrix, raw results, the manifest and checksums.

12. Approval

Name	Role	Signature	Date